How to make a good RL task

Eli Gottlieb
@GottliebEli · Oct 13

Follow

23 66 748 52K

I've been making lots of tasks lately. Many poor, some decent, and now finally, a few good. Through this work I've gained what I believe to be valuable insights into the task generation process that I'd like to share.

I'll break this up into four sections:

1. How I define a task
2. The inner-workings of a task
3. My view on making good tasks
4. Building task factories

What is a task?

I like to think of a task as four interconnected components:

Prompt

The request. For a coding task, it might be "Go implement feature X" or "I'm seeing bug Y, fix it." A finance task prompt might be "Here are some documents, here are some spreadsheets, go build me this financial model."

World

The data and dynamics that dictate the environment in which the model operates. For a coding task, the world might be a code repository; for a finance task, the world might be a bundle of a messy filesystem, excel, and a slack clone. A world can, but need not be, the real world.

(To interact with the world the model uses tools; for simplicity sake let's consider that set of tools, and the protocol through which they are exposed to the model, as sufficient and fixed)

Verifier

How we define what 'good' looks like. This can be a deterministic mechanism (a grading script to ensure tests pass on a piece of software) or dictated by a model (a single LLM/Jev call comparing a produced answer against some reference, or even an entirely new agent session to grade work).

Intent

The thing you want the model to get better at, named. Be it fixing a certain type of software bug, executing a bespoke workflow, or acting according to a specific doctrine, your task should be grounded in something explicit you want your model to improve at or a concrete failure mode you want to address.

Okay, so we got our prompt, world, verifier, and some task intent. How do these all connect?

Inner-workings of a task

I think of the task generation process as a constraint satisfaction problem: you, the task author, are fiddling with a set of interconnected components—your prompt, world, and verifier—while staying bound to some overarching intent. Fiddle too much with one component and you'll find yourself outside of the target distribution; fiddle too little and you might be reinforcing the wrong thing. That's all very vague and probably makes no sense, so let me define the important axes of alignment:

Prompt <-> Verifier

Your prompt and verifier should agree. If you consider your verifier fixed, your prompt should contain enough signal to guide a sufficiently intelligent model to satisfy whatever that verifier is. Say your task is to add multiplayer support to some basic python version of Pacman, and the verifier is a robust test suite that ensures proper multiplayer implementation against the produced code. If your task prompt is "make this game better," that's unfair—you're not giving the model enough of a chance to earn the reward.

Verifier <-> Intent

Your verifier should be aligned with the task intent. The verifier is the sole source of optimization pressure; whichever behaviors lead to its satisfying will be reinforced, including undesirable ones. You've probably heard of this misalignment between verifier and intent—"reward hacking": the model is able to achieve the reward without doing the thing you actually want it to do. Writing incomplete, buggy code that still passes all tests in a grading script is one example; escaping your sandbox and hacking Hugging Face to find the answer to your question is another.

Prompt <-> Intent

Your prompt must be aligned with the task intent. Say you want to train a chatbot that can intuitively maneuver through internal documents using a custom DSL to answer questions. When generating tasks for training, the prompts should be distributionally aligned to the types of questions the

chatbot will face in production. If we can expect the chatbot's users to bias toward a certain prompting style (i.e. underspecified queries), you should also adhere to that prompting style in training. Models are bad at talking like us (cool work to address this)—task prompts straying too far from what your agent in production will be faced with and you risk honing certain skills that won't be exercised in production.

Worlds

Worlds are highly task dependent and probably deserve a whole post of their own—they come in many shapes and sizes and live on a spectrum from 'real' to 'simulated': a coding task on an uncorrupted repo I'd consider a pretty 'real' world; a finance task that asks a model to produce a report over a corpus of real internal documents plus grounded synthetic auxiliary data is still a pretty 'real' world with a dash of 'synthetic'; a computer use task that requests the booking of an entire two week vacation across four fully cloned travel websites is pretty darn 'synthetic'.

There's benefits and drawbacks to living in each camp—going into more detail I worry is scope creep—but it's important to understand that for tasks existing in these semi- or fully synthetic worlds, the world itself is another fiddleable optimization surface that must align with its corresponding prompt, verifier, and intent (the closer your task's world is to the real world, the less of this fiddling you should need to do to it, given that the real world *is* the ground truth target).

A helpful framing of this problem for the semi- to fully synthetic worlds: a world is sufficient if and only if it can reliably elicit the task's target failure mode. Quick example: the synthetic "book me a two week vacation" task from above would be testing a model's long horizon reasoning, planning, and thoroughness in the web domain. Any failed attempt should exhibit a deficiency in one of these areas; successful attempts should exhibit mastery across these skills. The model stumbling over a bug in one of the synthetic websites is not a legitimate failure; the model producing the right answer while skipping what should have been a necessary step in the research process might mean the world is not rich enough to effectively back this task. This of course assumes a sound verifier and well calibrated prompt, highlighting the delicateness of the interplay between these components.

And why is all of this important, you may ask? Why must we hold ourselves to the highest of standards in this arduous endeavor? Reinforcement learning doesn't distinguish between intended and unintended behaviors; it simply reinforces whatever earns reward. Any misalignment in the axes above leaves you susceptible to reinforcing the wrong thing. While this may seem insignificant, it can have catastrophic effects—highlighted by the recent OpenAI Hugging Face attack where (one can surmise) a model was trained on tasks that had rewards which could be gamed through unkosher methods which led to misaligned, highly dangerous behavior being reinforced.

Now that we understand how these components connect, let's move to the fun stuff: how to actually make a task. And one more note I may not have made clear yet: of course we want coding agents to do as much of this work as possible!

My view on making good tasks

Making a good task boils down to identifying and implementing revision loops that can properly calibrate a task's optimization surfaces—its prompt, verifier, and world—against a legitimate task intent or failure mode.

I already said that I see the task generation process as a constraint satisfaction problem, where a task author is puppeteering a delicate dance across a task's interconnected components in pursuit of exposing legitimate model failures. This delicate dance can be encoded through pipelines of agents.

A simple task generation pipeline might look like:

1. author the task
2. eval the task
3. audit the traces
4. either loop back to the author (1) for modification, save the task as is, or discard the task

A slightly more complex SWE pipeline might look like:

1. author the task prompt (no verifier yet)
2. in parallel, give some agents ("solver agents") the task prompt and ask them to solve the task; launch another agent ("verifier agent") with the task prompt and ask it to create a grading script for the task
3. once the "solver agents" are done and the "verifier agent" has produced the grader, grade the solvers' work; also launch another set of agents ("hacker agents") with the task prompt + grading script + task intent and ask them to seek out an undesirable solution that still passes the tests
4. pass all of this info (the grading script, the solver agents' work + their scores, the hackers' findings) to a review agent who decides whether to save the task, drop the task, or loop back to either the original task author (1) or the verifier agent (2) for modifications

These pipelines are nothing more than natural language programs of agents doing work. The alpha lives in the recipe: the orchestration and order and specialization and looping that can wrangle a task and its interconnected components into shape.

A revision loop I've come across in almost all of my work is that of calibrating the verifier. Whether it's deterministic or LLM based, you need to be confident that your verifier is rewarding the good and punishing the bad. Rarely do you get this right on the first try; a proper revision loop must map out edge cases and regression test against ground truth labels before claiming a verifier is sound and faithful.

Similarly, taking an existing task and making it ‘harder’ can be a dangerous game: naively, revision loops aimed at increasing task difficulty will target artificial or contrived axes which don’t align with the task’s intent (like adding exhaustive requirements to a task prompt in an unrealistic way). A revision loop that makes tasks harder must be calibrated to scale difficulty along meaningful axes.

Okay, cool. So how do we do this at scale?

The verifiability spectrum

A quick step back—in my discussion of “legitimate task failures,” “meaningful difficulty axes,” and “task intent,” I've neglected to name an important piece of this all: task verifiability. I see this as a spectrum—on one end, you have a highly verifiable math task with a single correct answer; on the other end, a creative writing task with no clear definition of ‘right’ or ‘good’.

The vast majority of interesting work in our world lives somewhere in the middle: there’s no single ‘right way’ to do research, law work, finance work, or work in pretty much any other white collar industry. Even domains which might seem verifiable like coding have important, non-verifiable axes, such as taste, style, and readability. For these non-verifiable features you want to optimize against, there has to be some source of signal, some definition of a “job well done.” In my opinion, for the vast majority of domains, that source of signal will still be human.

The investment banker who understands the nuances of how the financial model should be produced. The software engineer who wants to imbue the model with good engineering taste. These human experts are the ultimate verifiers.

As a thought exercise, consider a world where you could perfectly digitize a human expert and have them inspect each component of this pipeline. How would you prompt them? What would you have them produce? You’d likely have them scrutinize task prompts, analyze tool calls to understand the dynamics of the world, and audit the verifiers to ensure their definition of ‘good’ is being properly approximated.

This world is obviously fictitious, but serves as a valuable subproblem to solve for when mapping out how a task pipeline should be organized.

Task factories

A task factory is an outer optimization loop that progressively distills human judgement into the task generation pipeline itself, continually improving its ability to produce high quality tasks aligned with human intent at progressively lower cost.

We now have two optimization loops:

Inner: the aforementioned recipe. A pipeline of agents operating over a task's optimization surfaces (prompt, world, verifier). This recipe is bound to some overarching intent—for example, producing finance tasks that test whether an agent can independently identify relevant source documents, reconcile conflicting figures, and construct a sound financial model.

Outer: a mechanism for updating the recipe as our understanding of what constitutes a good task sharpens.

Example: while inspecting an initial set of finance tasks, the investment banker realizes the task prompts are unrealistically guiding the model to the specific files it needs to read—in the real world there would be no such hint. This outer loop takes that authoritative signal and updates the recipe so future tasks more faithfully serve the original objective.

I defined a recipe as a natural language program of agents doing work: "Author the task, then eval, then audit, if issues route back to author, if sound save the task, if unsalvageable drop it."

How one actually encodes this recipe, and the amount of structure imposed on it, I think is up for debate. On one end of the spectrum, your recipe could be a rigid script authored in some agent SDK; on the other end it could be a single skill file, handed to a system like Prime Agent that is capable of orchestrating arbitrarily complex swarms of subagents. I believe that, given current model capabilities and costs, the optimal substrate lies somewhere in the middle: a state machine that harnesses general agentic capabilities within a higher-level orchestration structure.

However you choose to build your recipe, that choice will dictate the optimization surfaces this outer loop operates over. With minimal structure, maybe its just a single skill file that gets continually updated; in this middle ground that I bias towards, the surfaces are the order in which agents run, the modularization of duty, how each agent is conditioned, the way information flows through the system, etc.

A few strategies I've found helpful for structuring this outer loop:

Decompose human judgement into explicit criteria

For every task family, name the important features. These might include:

- task prompts aligned with production traffic
- failed attempts demonstrate a meaningful deficiency you want the task to target
- successfully attempts exhibit the behavior you want to reinforce
- worlds contain sufficient information and complexity to meaningfully exercise the targeted skill

These are your protected axes of task quality. For each axis, implement a guard in the pipeline that checks whether a generated task satisfies it. Example: a finance task authoring agent might be followed by a review agent tasked specifically with identifying unrealistic hints in the prompt—references to particular files, sheets, or figures that a real analyst wouldn't have been given. If it finds any, the task gets routed back to the author for revision.

Compile human judgement into progressively cheaper recipe components

Each component of your recipe does *something*—you might have a task authoring component, a grading component, an auditing component, you get the idea. A task pipeline should maximize quality yield while minimizing dollar spend per task, so as time goes on and signal is accumulated in the currency of decisions, explanations, and annotations from a trusted source, there should exist a compilation algorithm that can develop cheaper substitutes that sufficiently preserve the judgement/capabilities of the more expensive component.

Take the finance example from above. Initially, the investment banker might literally serve as a gate in the recipe, dropping bad tasks, confirming good ones, and annotating those that need improvement. Eventually, after accumulating enough examples, this component can be replaced with an agent session proven to sufficiently proxy the human expert. For some components, such as a task prompt gate, this compilation, assuming proper capability preservation assurances, can go all the way down to a cheap classifier like Jev.

What does this give us?

This is continual learning applied to the task factory: feedback on individual tasks is compounded into the recipe, so the factory gets progressively better at producing good tasks without repeatedly paying to relearn the same lessons. And, if set up properly, this signal can be used to train the data generation models that operate the factory itself.

I see no reason why this approach shouldn't generalize across domains provided we design the right interfaces for extracting human judgment and incorporating that expertise into the pipelines themselves. And the beauty of it is that as frontier models become more capable, they only get better at

internalizing and distilling this expert judgment, enabling them to be more successful puppeteers of the entire system.

The goal is to effectively extract signal in semi-verifiable domains and distill that signal into a hill climbing machine. I think we're close.

Thanks to @brendanh0gan, @Grad62304977, @MikeRCrystal and Artem Zhuravel for feedback!

Want to publish your own Article?

Upgrade to Premium

Eli Gottlieb
@GottliebEli
interested in ai + philosophy

Follow